



INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIAS E CIÊNCIAS
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIAS
CURSO DE ENGENHARIA INFORMÁTICA

MANUAL DO UTILIZADOR

COMPILADORES

Nome: Abel Nkele Canas

Nº Processo: 20231051

Turma: M3

Ano: 3º ano

ÍNDICE

ÍNDICE	2
1. O QUE É ESTE PROGRAMA	4
2. REQUISITOS	5
3. INSTALAÇÃO	5
Opção A — Abrir no NetBeans (mais simples)	5
Opção B — Compilar manualmente pela linha de comandos	5
4. COMO EXECUTAR	5
Sobre acentos na consola do Windows	5
5. FORMATO DO FICHEIRO DE ENTRADA	6
6. COMO LER A SAÍDA DO PROGRAMA	6
6.1. Nota para quem quiser depurar o Lexer (avanzado)	6
6.2. A Tabela de Símbolos	7
6.3. Erros sintáticos vs. erros semânticos	7
7. SUBCONJUNTO DA LINGUAGEM SUPTADO	8
O que o analisador semântico verifica, especificamente	8
8. EXEMPLOS COMPLETOS PASSO A PASSO	9
8.1. Exemplo sem erros	9
8.2. Exemplo com erros semânticos (todas as verificações)	9
8.3. Exemplo com arrays e for-each	10
8.4. Exemplo com erro sintático	11
9. MENSAGENS DE ERRO MAIS COMUNS E O QUE SIGNIFICAM	11
10. PERGUNTAS FREQUENTES	12
11. LIMITAÇÕES E O QUE O PROGRAMA NÃO FAZ	12
12. RESOLUÇÃO DE PROBLEMAS (TROUBLESHOOTING)	13
13. GLOSSÁRIO DE TERMOS TÉCNICOS	13
14. REFERÊNCIA RÁPIDA: PALAVRAS-RESERVADAS E OPERADORES RECONHECIDOS	15
14.1. Palavras-reservadas	15
14.2. Literais especiais	15
14.3. Operadores e delimitadores	15
14.4. Categorias de literais	16

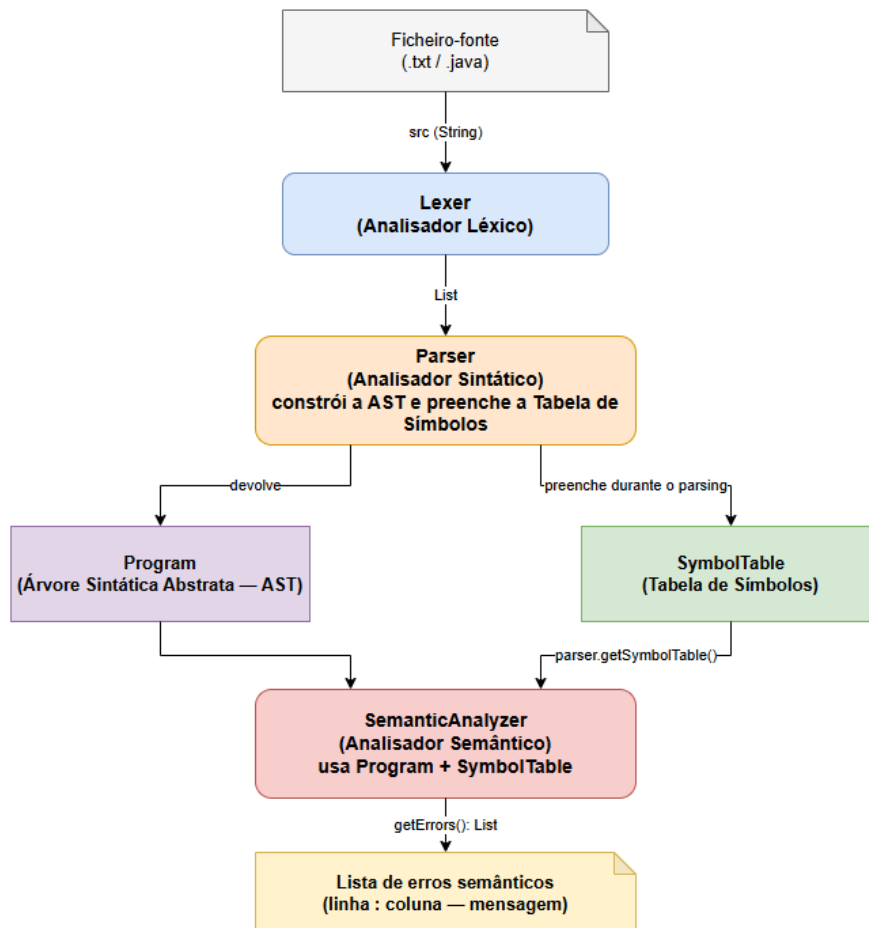
15. Conclusão	17
16. REFERÊNCIAS BIBLIOGRÁFICAS.....	18

1. O QUE É ESTE PROGRAMA

Este MINI-COMPILADOR é uma ferramenta educativa que analisa ficheiros de texto escritos num **subconjunto da linguagem Java** e verifica se estão corretos em três níveis, um a seguir ao outro:

1. **Léxico** — se todos os "pedaços" do texto (palavras, números, símbolos) são reconhecidos.
2. **Sintático** — se esses pedaços estão organizados de acordo com a gramática da linguagem (chavetas fechadas, ponto-e-vírgula no sítio certo, etc.).
3. **Semântico** — se o programa "faz sentido": variáveis usadas foram declaradas, os tipos são compatíveis entre si, os métodos são chamados com o número e tipo de argumentos certos, etc.

O programa **não executa** o código Java nem gera um programa final — o seu único objetivo é **detetar e reportar erros**, indicando sempre a linha e a coluna onde cada problema ocorre, tal como um compilador real faz antes de gerar código.



2. REQUISITOS

- **Java Development Kit (JDK) versão 17 ou superior** instalado no computador.
- Um terminal (PowerShell, cmd, ou o terminal do NetBeans/VS Code).
- Um editor de texto simples para escrever os ficheiros de entrada (Bloco de Notas, Notepad++, VS Code, ou o próprio NetBeans).

Para confirmar que o Java está instalado, abra um terminal e execute:

```
java -version
```

Deve aparecer algo como `java version "17..."` ou superior.

3. INSTALAÇÃO

Não é necessária "instalação" no sentido tradicional — este é um projeto de código-fonte Java. Existem duas formas de o preparar:

Opção A — Abrir no NetBeans (mais simples)

1. Abrir o NetBeans.
2. `File → Open Project...` e seleccionar a pasta `Compiler/`.
3. Clicar com o botão direito no projeto → **Build**.

Opção B — Compilar manualmente pela linha de comandos

Dentro da pasta `Compiler/`, execute:

```
Get-ChildItem -Recurse -Filter *.java src | ForEach-Object { $_.FullName } | Out-File  
sources.txt -Encoding utf8  
javac -d build\classes -encoding UTF-8 "@sources.txt"
```

Isto cria os ficheiros `.class` compilados dentro de `build\classes`.

4. COMO EXECUTAR

Depois de compilado, a partir da pasta onde ficaram as classes (por exemplo `build\classes`, ou `Compiler\build\classes` se usou o NetBeans):

```
java com.compiler.main.Main "caminho\para\o\seu\ficheiro.txt"
```

Por exemplo, para analisar um dos ficheiros de exemplo incluídos no projeto:

```
java com.compiler.main.Main "..\..\teste123.txt"
```

Se não indicar nenhum ficheiro, o programa tenta abrir por omissão um ficheiro chamado `teste_de_erro.txt` na pasta atual.

Sobre acentos na consola do Windows

Como as mensagens do programa estão em português (com "á", "ç", "õ", etc.), a consola do Windows pode, por predefinição, mostrar esses caracteres incorretamente

(ex.: **semântico** em vez de **semântico**). Para evitar isto, antes de correr o programa, execute uma vez por sessão de terminal:

```
chcp 65001
```

E execute o programa incluindo os parâmetros de codificação:

```
java "-Dfile.encoding=UTF-8" "-Dstdout.encoding=UTF-8" com.compiler.main.Main  
"ficheiro.txt"
```

5. FORMATO DO FICHEIRO DE ENTRADA

O ficheiro de entrada é um ficheiro de texto simples (extensão **.txt** ou **.java**, tanto faz — o programa só lê o conteúdo) que deve conter **uma classe ou interface Java** (ou várias, uma a seguir à outra), escrita seguindo as regras descritas na secção 7.

Exemplo mínimo válido:

```
public class Ola {  
    public static void main(String[] args) {  
        int x = 10;  
        System.out.println(x);  
    }  
}
```

6. COMO LER A SAÍDA DO PROGRAMA

Ao correr o programa, a saída aparece organizada em secções, sempre pela mesma ordem:

```
=== ERROS SEMANTICOS ===      (só aparece se houver erros)  
- lista de problemas semânticos encontrados, no formato:  
  "Erro semântico em <linha>:<coluna> - <descrição>"  
  
✓ COMPILAÇÃO CONCLUÍDA COM SUCESSO!      (a verde, se não houver  
erros)  
ou  
✗ COMPILAÇÃO CONCLUÍDA COM ERROS!      (a vermelho, com a contagem  
total)
```

Esta é a saída **focada no Analisador Semântico**, que é o objeto de avaliação deste exame: o programa mostra o resultado do parser, a Tabela de Símbolos completa (com todos os atributos exigidos pelo enunciado — endereço, tipo, dimensão, valor, escopo, tamanho em bytes) e a lista de erros semânticos, sem reimprimir o código-fonte ou o fluxo de tokens do Lexer, que não fazem parte do que este componente precisa de demonstrar.

6.1. Nota para quem quiser depurar o Lexer (avanzado)

O código de **Main.java** mantém, comentados (não apagados), os blocos que reimprimiam o código-fonte lido e listavam cada token reconhecido pelo Lexer, um por

linha, no formato Token[TIPO | 'lexema' | linha:coluna]. Se precisar de depurar especificamente a fase léxica, basta descomentar esses blocos em Main.java — a lista de tokens já é calculada internamente de qualquer forma, pois o Parser precisa dela; apenas a impressão está desativada por omissão.

6.2. A Tabela de Símbolos

Esta tabela lista **todas as declarações** encontradas no programa (classes, campos, parâmetros, variáveis locais, métodos, construtores), com estas colunas:

Coluna	Significado
Token	Categoria do símbolo (CLASS, FIELD, METHOD, PARAMETER, LOCAL_VARIABLE, ...)
Lexema	O nome do identificador
Linha	Linha onde foi declarado
Categoria	Descrição por extenso (ex.: "variável local")
Tipo de dado	int, String, MinhaClasse, etc.
Tipo de variável	Se é atributo, parâmetro ou variável local
Escopo	Onde a variável "vive" (ex.: MinhaClasse.metodo)
Valor	Valor inicial atribuído, se existir
Endereço	Endereço de memória simulado (fins didáticos, não é memória real)
Tamanho	Tamanho em bytes do tipo
Inicializado	true/false
Dim.	Número de dimensões, se for um array (int[][] = 2)
Parâmetros	Lista de parâmetros, se for um método/construtor
Modificadores	public, static, final, etc.
Retorno	Tipo de retorno, se for um método

6.3. Erros sintáticos vs. erros semânticos

- Um **erro sintático** significa que o código está "mal escrito" segundo a gramática (ex.: falta um ;, uma chaveta não foi fechada). Estes erros são detetados primeiro.
- Um **erro semântico** significa que o código está bem escrito, mas não faz sentido logicamente (ex.: usar uma variável nunca declarada, comparar tipos incompatíveis, chamar um método com argumentos errados). Este programa

continua a analisar semanticamente mesmo que já existam erros sintáticos noutra parte do ficheiro, para dar o máximo de informação possível numa só execução.

7. SUBCONJUNTO DA LINGUAGEM SUPORTADO

O compilador aceita um subconjunto amplo e realista de Java. Em resumo:

Estrutura do programa: package, import (incluindo import static e import x.y.*), declaração de classes e interfaces (com modificadores public, abstract, final, static, etc.), classes/interfaces aninhadas, parâmetros de tipo genérico na sintaxe (<T extends X>), extends e implements.

Membros de classe: campos (com múltiplos declaradores e inicializadores), métodos (com corpo ou abstratos), construtores, blocos de inicialização (estáticos ou de instância).

Tipos: todos os tipos primitivos (boolean, byte, short, char, int, long, float, double), String, void, tipos definidos pelo utilizador, arrays de qualquer dimensão (int[], String[], ...).

Instruções: declaração de variáveis locais, if/else (incluindo encadeado), while, do-while, for (clássico e for-each), switch (com case/default), break, continue, return, throw, try/catch/finally, assert, synchronized, blocos { }, instrução vazia ;.

Expressões: todos os operadores aritméticos, relacionais, lógicos, bit a bit, de atribuição (simples e composta), instanceof, ? (ternário), incremento/decremento (pré e pós-fixos), cast (Tipo) expr, acesso a membros (objeto.campo, objeto.metodo(...)), acesso a arrays (array[indice]), criação de objetos (new Classe(...)), criação de arrays (new int[10], com inicializador {1,2,3}), literais (inteiro, long, float, double, texto, caractere, true/false/null), this, super, comentários de linha (//) e de bloco (/* */), com aninhamento).

O que o analisador semântico verifica, especificamente

De acordo com o exigido para este exame, o programa verifica sempre:

1. **Variáveis não declaradas** — usar um nome que nunca foi declarado no escopo acessível.
2. **Declarações duplicadas** — declarar duas vezes o mesmo nome no mesmo escopo (ex.: dois campos total na mesma classe).
3. **Incompatibilidade de tipos** — usar operadores com tipos que não fazem sentido juntos (ex.: somar um boolean com um int).
4. **Tipos dos argumentos em chamadas de métodos** — número e tipo dos argumentos passados numa chamada devem coincidir com os parâmetros declarados.
5. **Atribuições incompatíveis** — atribuir um valor de um tipo a uma variável de outro tipo incompatível (ex.: atribuir uma String a um int).

6. **Condições das estruturas de controlo** — a condição de `if`, `while`, `do-while`, `for` e `assert` deve ser sempre do tipo `boolean`; a expressão de um `switch` deve ser `int/char/String`; o tipo do elemento de um `for-each` deve ser compatível com o array percorrido.

8. EXEMPLOS COMPLETOS PASSO A PASSO

8.1. Exemplo sem erros

Ficheiro `ola.txt`:

```
public class Ola {  
    public static void main(String[] args) {  
        int x = 10;  
        int y = 20;  
        if (x < y) {  
            System.out.println("x é menor que y");  
        }  
    }  
}
```

Ao correr `java com.compiler.main.Main ola.txt`, a saída termina com:

```
Parser executado com sucesso!  
...  
Analizador semântico executado sem erros!  
  
✓ COMPILAÇÃO CONCLUÍDA COM SUCESSO!  
✓ Nenhum erro encontrado durante a análise.
```

8.2. Exemplo com erros semânticos (todas as verificações)

Ficheiro `teste_semantico.txt` (incluído no projeto):

```
public class TesteSemantico {  
    private int total = 0;  
    private int total = 5;  
  
    public int soma(int a, int b) {  
        return a + b;  
    }  
  
    public void executar() {  
        int x = 10;  
        String texto = "ola";  
  
        int y = x + naoDeclarada;  
  
        int z = texto;  
  
        int resultado = soma(1, "dois");  
    }  
}
```

```

    if (x) {
        System.out.println("nunca deveria compilar");
    }

    while (texto) {
        System.out.println("laço inválido");
    }

    for (int i = 0; texto; i++) {
        System.out.println(i);
    }
}

```

Ao correr o compilador sobre este ficheiro, a secção de erros semânticos mostra:

- Erro semântico em 3:25 - Símbolo 'total' já declarado no escopo 'TesteSemantico'
- Erro semântico em 13:21 - Variável 'naoDeclarada' não declarada
- Erro semântico em 15:17 - Não é possível atribuir um valor do tipo 'String' para a variável 'z' do tipo 'int'
- Erro semântico em 17:33 - Argumento 2 de 'soma' tem tipo 'String', esperado 'int'
- Erro semântico em 19:13 - Condição de 'if' deve ser do tipo boolean, mas é 'int'
- Erro semântico em 23:16 - Condição de 'while' deve ser do tipo boolean, mas é 'String'
- Erro semântico em 27:25 - Condição de 'for' deve ser do tipo boolean, mas é 'String'

X COMPILAÇÃO CONCLUÍDA COM ERROS!

X 7 erro(s) encontrado(s) durante a análise.

Repare como cada uma das seis exigências do enunciado está representada: duas declarações do campo `total` (b), uso de `naoDeclarada` (a), atribuição de `String` a `int` (c/e), argumento `"dois"` onde se esperava `int` (d), e três condições de controlo que não são `boolean` (f).

8.3. Exemplo com arrays e for-each

Ficheiro `array_exemplo.txt`:

```

public class ArrayExemplo {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3, 4, 5};

        for (int n : numeros) {
            System.out.println(n);
        }

        int soma = 0;
        for (int i = 0; i < numeros.length; i++) {
            soma = soma + numeros[i];
        }
        System.out.println(soma);
    }
}

```

```
}
}
```

Este exemplo está correto: o `for-each` percorre `numeros` (um `int[]`) com uma variável de laço `n` do tipo `int`, que é compatível com o tipo do elemento do array; o acesso `numeros[i]` usa um índice `int`, tal como exigido. A tabela de símbolos gerada mostra `numeros` com `Dim. = 1` (um array de uma dimensão) e o compilador termina sem erros.

Se, em vez disso, a variável do `for-each` fosse declarada com um tipo incompatível — por exemplo `for (String n : numeros)` — o compilador reportaria:

- Erro semântico em 5:22 - Elemento do tipo 'int' incompatível com a variável 'n' (String) do for-each

8.4. Exemplo com erro sintático

Ficheiro `teste_de_erro.txt` (incluído no projeto), propositadamente malformado (falta um `;`, um valor de atribuição e a classe nunca abre um bloco correto):

```
public class TesteErros
    private int x = ;
    private String nome
    private int y = 10
}
```

A saída mostra `Parser terminou com erro(s)`, mas recuperou e continuou a análise. seguido da lista de problemas sintáticos encontrados, cada um com linha e coluna — o programa não para na primeira ocorrência, tenta sincronizar e continuar a analisar o resto do ficheiro.

9. MENSAGENS DE ERRO MAIS COMUNS E O QUE SIGNIFICAM

Mensagem (exemplo)	O que significa	Como corrigir
Variável 'x' não declarada	Usou um nome que nunca foi declarado antes, nesse escopo	Declarar a variável antes de a usar, ou corrigir um erro de escrita no nome
Símbolo 'x' já declarado no escopo 'Y'	Duas declarações com o mesmo nome, no mesmo escopo	Remover ou renomear uma das duas declarações
Operador '+' não pode ser aplicado a tipos incompatíveis 'X' e 'Y'	Está a tentar usar um operador (+, -, &&, etc.) com tipos que não são compatíveis entre si	Verificar os tipos das duas variáveis/expressões envolvidas
Não é possível atribuir um valor do tipo 'X' para a variável 'v' do tipo 'Y'	O tipo do valor à direita do = não é compatível com o tipo declarado da variável	Corrigir o tipo da variável ou o valor atribuído
Argumento N de 'metodo' tem tipo 'X', esperado 'Y'	Um dos argumentos passados na chamada não tem o tipo esperado pelo	Verificar a ordem e o tipo dos argumentos passados

	parâmetro correspondente	
Número de argumentos incorreto para 'metodo': esperado N, encontrado M	Chamou um método/construtor com um número de argumentos diferente do declarado	Verificar quantos parâmetros o método/construtor espera
Condição de 'if'/'while'/'for' deve ser do tipo boolean, mas é 'X'	A condição de uma estrutura de controlo não é uma expressão boolean	Trocar por uma comparação ($x > 0$, $x == 5$, etc.) em vez do valor "cru"
Método 'x' não existe na classe 'Y'	Está a chamar um método que não foi declarado nessa classe	Verificar o nome do método ou se a classe está definida no ficheiro
Campo 'x' não existe na classe 'Y'	Está a aceder a um campo que não foi declarado nessa classe	Verificar o nome do campo

10. PERGUNTAS FREQUENTES

P: Posso analisar vários ficheiros `.java` ao mesmo tempo? R: Não. O programa recebe um único caminho de ficheiro como argumento. Para analisar vários programas, execute o comando uma vez por ficheiro.

P: O programa executa o meu código Java? R: Não. Este é apenas o *front-end* de um compilador (análise léxica, sintática e semântica) — não há geração nem execução de código.

P: Porque é que dois construtores com o mesmo nome (sobrecarregados) dão erro de "já declarado"? R: Este mini-compiler, por desenho, não modela sobrecarga de métodos ou construtores — cada nome só pode ter uma assinatura por escopo (ver secção 11). Se precisar de comportamentos diferentes, use nomes de métodos diferentes.

P: Os meus `imports` de bibliotecas de terceiros dão erro? R: O programa não valida o conteúdo de imports — apenas os reconhece sintaticamente. Classes comuns do Java (System, String, ArrayList, Math, etc.) são reconhecidas por uma lista interna; classes verdadeiramente externas ao ficheiro-fonte não são validadas quanto aos seus membros.

P: Como sei se um erro é do analisador léxico, sintático ou semântico? R: Pela secção onde aparece na saída (ERROS SINTATICOS ou ERROS SEMANTICOS) — erros léxicos aparecem como avisos na consola ([AVISO] Caractere não reconhecido...) durante a fase de tokens.

11. LIMITAÇÕES E O QUE O PROGRAMA NÃO FAZ

- Não gera código executável nem um programa final — é apenas um verificador.
- Não modela sobrecarga de métodos/construtores (dois métodos com o mesmo nome no mesmo escopo são tratados como duplicados).

- Não modela herança entre classes definidas pelo próprio utilizador (apenas `Object` é tratado como super-tipo universal).
- Não valida os membros de classes externas à biblioteca padrão conhecida (apenas aceita o nome da classe).
- Não deteta erros de fluxo (ex.: "esta variável pode não ter sido inicializada neste caminho", "falta um `return` nalgum ramo").
- Os endereços de memória apresentados na tabela de símbolos são **simbólicos e sequenciais**, apenas para fins didáticos — não representam posições reais de memória.

12. RESOLUÇÃO DE PROBLEMAS (TROUBLESHOOTING)

Sintoma	Causa provável	Solução
Erro no parser: ... na consola e nada mais é impresso	Uma exceção inesperada interrompeu a análise (ex.: ficheiro vazio, ou combinação de sintaxe não coberta)	Confirmar que o ficheiro não está vazio e que é um Java válido de alto nível; testar com um dos ficheiros de exemplo primeiro
Erro <code>NoSuchFileException</code> ou semelhante	O caminho do ficheiro passado como argumento está incorreto	Confirmar o caminho (relativo à pasta onde está a correr o java)
Caracteres acentuados aparecem como ? ou símbolos estranhos	Codificação da consola não está em UTF-8	Executar <code>chcp 65001</code> antes, e correr o java com <code>-Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8</code> (ver secção 4)
[AVISO] Caractere não reconhecido: 'X'	Existe no ficheiro-fonte um carácter que não pertence a nenhuma categoria de token (ex.: um emoji, ou um símbolo de outra linguagem)	Remover ou substituir o carácter no ficheiro-fonte
Uma classe/método que existe no ficheiro é reportado como "não declarada"	O identificador está a ser usado antes da sua classe estar registada corretamente, ou há um erro de escrita (maiúsculas/minúsculas importam em Java)	Confirmar a grafia exata (Java é <i>case-sensitive</i>) e que a classe está definida no mesmo ficheiro
O programa não compila (javac falha)	Versão do JDK inferior a 17	Instalar/usar o JDK 17 ou superior

13. GLOSSÁRIO DE TERMOS TÉCNICOS

Termo	Significado, em linguagem simples
-------	-----------------------------------

Token	O menor "pedaço" reconhecível de um programa: uma palavra, um número, um símbolo. Ex.: int, x, =, 10, ; são cada um um token separado.
Lexema	O texto exato de um token, tal como aparece no ficheiro (ex.: o lexema do token "identificador" pode ser contador).
Analizador Léxico (Scanner/Lexer)	A parte do programa que lê o ficheiro carácter a carácter e o transforma numa lista de tokens.
Analizador Sintático (Parser)	A parte do programa que verifica se a sequência de tokens está organizada segundo as regras gramaticais da linguagem, construindo uma árvore que representa essa estrutura.
Árvore Sintática Abstrata (AST)	A representação em forma de árvore da estrutura do programa, produzida pelo analisador sintático (por exemplo, um if é representado como um nó com um ramo para a condição, um ramo para o "então" e um ramo opcional para o "senão").
Analizador Semântico	A parte do programa que verifica se o programa, apesar de gramaticalmente correto, também faz sentido (tipos compatíveis, variáveis declaradas, etc.).
Escopo	A "zona" do programa onde um nome (variável, parâmetro, etc.) está visível e pode ser usado. Por exemplo, uma variável declarada dentro de um método só é visível dentro desse método.
Tabela de Símbolos	Uma espécie de "ficha" com todas as declarações do programa (nome, tipo, escopo, endereço, etc.), usada pelo compilador para saber tudo sobre cada identificador.
Tipo (Type)	A categoria de um valor: int, boolean, String, uma classe do utilizador, um array, etc. Duas expressões só podem normalmente combinar-se (somar, comparar, atribuir) se os seus tipos forem compatíveis entre si.
Compatibilidade de tipos / atribuição (widening)	A regra que diz quando é seguro "converter automaticamente" um valor de um tipo para outro (por exemplo, um int pode ser atribuído a um double sem perda de informação, mas o inverso não

	é automático).
Erro léxico	Um problema detetado enquanto o programa ainda está a ser dividido em tokens (ex.: uma string que nunca fecha as aspas).
Erro sintático	Um problema de estrutura/gramática (ex.: falta um ; ou uma chaveta).
Erro semântico	Um problema de "significado" do programa (ex.: usar uma variável nunca declarada, ou somar um texto com um booleano).
Modo pânico (recuperação de erros)	A estratégia usada pelo compilador para, ao encontrar um erro, não parar imediatamente: avança no texto até um ponto seguro (normalmente um ; ou um }) e continua a analisar o resto do programa, reportando o máximo de erros possível numa só execução.

14. REFERÊNCIA RÁPIDA: PALAVRAS-RESERVADAS E OPERADORES RECONHECIDOS

14.1. Palavras-reservadas

```
abstract assert boolean break byte case catch
char class const continue default do double
else enum extends final finally float for
goto if implements import instanceof int
interface long native new package private
protected public return short static strictfp
super switch synchronized this throw throws
transient try void volatile while
```

(As palavras `const` e `goto` são reconhecidas como reservadas — tal como em Java — mas não têm nenhuma construção gramatical associada, exatamente como na própria linguagem Java.)

14.2. Literais especiais

`true`, `false`, `null`

14.3. Operadores e delimitadores

Categoria	Símbolos reconhecidos
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
Incremento/decremento	<code>++</code> <code>--</code>

Atribuição	= += -= *= /= %= &= = ^= <<= >>= >>>=
Relacionais	< > <= >= == !=
Lógicos	&& !
Bit a bit	& ^ ~
Deslocamento (shift)	<< >> >>>
Ternário	? :
Reticências (varargs)	...
Delimitadores	() { } [] ; , .

14.4. Categorias de literais

Categoria	Exemplos
Inteiro	0, 42, 1000
Long	42L, 1000l
Float	3.14f, 1.0F
Double	3.14159, 2.0d
Texto (String)	"Olá mundo!"
Caractere	'a', '\n'
Booleano / nulo	true, false, null

15. Conclusão

Este manual cobriu tudo o que é necessário para instalar, executar e interpretar corretamente os resultados do MINI-COMPILADOR: desde a preparação do ambiente (JDK 17+), passando pelo formato esperado do ficheiro de entrada e pela leitura de cada secção da saída (tokens, tabela de símbolos, erros sintáticos e semânticos), até aos exemplos práticos que ilustram cada uma das verificações semânticas implementadas. Com esta informação, qualquer utilizador — mesmo sem conhecimento prévio da implementação interna do compilador — consegue escrever um programa no subconjunto de Java suportado, correr o compilador sobre esse ficheiro, e compreender exatamente o que cada mensagem de erro reportada significa e como a corrigir.

16. REFERÊNCIAS BIBLIOGRÁFICAS

GOSLING, James; JOY, Bill; STEELE, Guy; BRACHA, Gilad; BUCKLEY, Alex; SMITH, Daniel; BIERMAN, Gavin. *The Java Language Specification*, edição correspondente ao Java SE 17. Oracle America, Inc.

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. 2ª edição. Boston: Pearson/Addison Wesley, 2006.

Documentação oficial do JDK (Java Development Kit), Oracle. Disponível em: docs.oracle.com/en/java.